SIMATS ENGINEERING
ASSESSMENT TOOL 3: Application Based Questions

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1516

COURSE OUTCOME COVERED
CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments. (BL3)

Assessment Tool Weightage: 10%

Dave's Taxonomy: Manipulation (Level 2) Total Marks: 25
Question Count: 5

Code of Conduct
I Tharunkumar.S / 192511416 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: 

Assessment Tasks

Application Based Questions

1. A company experiences unpredictable traffic spikes during seasonal sales. Explain how cloud auto-scaling and load balancing can handle this demand efficiently. How does this approach reduce downtime and improve user experience? What role do monitoring tools play in this setup? Relate your answer to a real-world e-commerce scenario.

2. An organization wants to store large volumes of backup data securely in the cloud.
   Discuss how cloud storage solutions ensure durability and data redundancy.
   How can encryption and access control protect sensitive backup data?
   Explain the cost advantages compared to traditional storage systems.
   Provide an example of a disaster recovery use case.
3. A mobile app company plans to deploy its application globally. Explain how content delivery networks (CDNs) improve performance and reduce latency. How do edge servers enhance user experience across regions? Discuss how cloud providers support global availability. Relate this to a real-time streaming or gaming application.

4. A financial institution requires strict compliance and data security in cloud usage.
   Explain how cloud providers support regulatory compliance and auditing. What role do logging, monitoring, and encryption play in security? How can the company ensure data sovereignty requirements are met? Give an example involving sensitive financial transactions.
5. A development team wants to adopt containerization for application deployment.
   Explain how containers differ from virtual machines in terms of performance and resource usage. How does orchestration (like Kubernetes) simplify deployment and scaling? Discuss portability benefits across different cloud environments.
   Provide an example of a microservices-based application.

Question 1

Problem interpretation and design requirements

QUESTION 1 A company experiences unpredictable traffic spikes during seasonal sales. Explain how cloud auto-scaling and load balancing can handle this demand efficiently. How does this approach reduce downtime and improve user experience? What role do monitoring tools play in this setup? Relate your answer to a real-world e-commerce scenario.

ANALYSIS FOCUS / ELASTIC E-COMMERCE INFRASTRUCTURE

Scenario and operating assumptions

A seasonal campaign creates sudden bursts of browsing, cart updates and checkout requests. A fixed server fleet either becomes saturated during the sale or stays underused outside peak hours. The architecture must accommodate sharp demand changes while protecting orders, stock consistency and payment processing.

The main success criterion is application-level reliability, not CPU utilization alone. The design should define availability and latency objectives, capacity headroom, an acceptable checkout error rate and a safe degradation mode when supporting services are overloaded.

Required cloud capabilities

Horizontal auto-scaling adds or removes application replicas from a defined policy. Layer-7 load balancing distributes requests only to healthy replicas and stops routing to failed instances. Multi-availability-zone placement reduces the impact of a single datacenter fault. CDN caching and a managed cache absorb static and read-heavy requests before they reach the application.

An order queue isolates checkout acceptance from slower downstream processing. The inventory or payment database requires transaction controls and idempotency so a retry does not create a second order or charge. The application tier should be stateless so any healthy replica can process a request.

Design goals

Keep user sessions consistent through shared session storage or secure tokens; avoid sticky-session dependence where possible. Monitor end-to-end latency and business failures, not merely infrastructure health. Use rate limiting and bot controls so malicious or noncritical requests cannot exhaust scarce checkout capacity.

Engineering evidence / comparison

Requirement	Concrete acceptance evidence
Peak capacity	Load test at forecast peak, plus a documented safety margin
Availability	Cross-AZ routing and tested failed-instance removal
Checkout correctness	One payment/order per idempotency key
Customer experience	Track p95 page and checkout latency, error rate and abandonment

DECISION
Use stateless services across multiple availability zones with target-based scaling and a health-aware load balancer. Scale background workers separately from web replicas.

Question 1

Reference architecture and request path

ANALYSIS FOCUS / PRODUCTION-READY SCALING TOPOLOGY

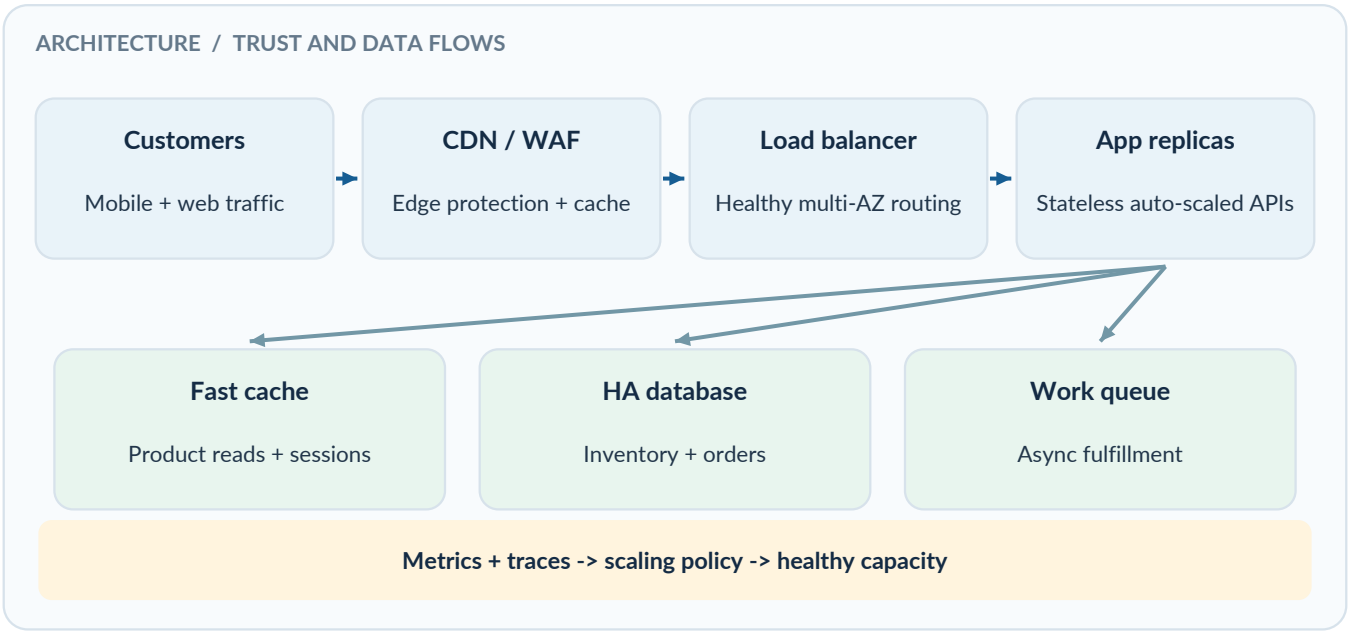


FIGURE / Cloud reference architecture - illustrates principal dependencies and control flow; provider-neutral design.

How requests travel

A customer reaches the CDN and web application firewall. Cacheable product images and static assets are delivered from the edge. Dynamic cart and checkout traffic is forwarded to the regional load balancer, which chooses a healthy service replica in an availability zone.

The application reads frequently requested product information from cache; authoritative stock and payment state remains in transactional storage. Queue-backed workers process emails, invoices and other asynchronous steps, without letting an email outage block the checkout response.

Failure and isolation boundaries

If one replica fails, the load balancer removes it after failed readiness or health checks. If an availability zone becomes unavailable, sufficient replicas must survive in the remaining zones; merely spreading a minimal replica count across zones does not guarantee failure capacity.

The database, cache and queue each need their own availability plan. Replicating stateless web servers alone does not prevent downtime if checkout depends on a single unprotected database instance.

ARCHITECTURE NOTE
Dashed connections indicate telemetry and scaling control; solid connections are principal request or data flows. Design represents a cloud-neutral reference implementation.

Question 1

Auto-scaling mechanics and capacity planning

ANALYSIS FOCUS / METRIC SELECTION, RULES AND TUNING

Scale-out policy

Choose a primary demand metric such as requests per target, CPU combined with latency, or queue age for worker pools. Configure a minimum healthy replica count, a maximum bounded by budget or downstream capacity, instance warm-up behavior and hysteresis to prevent oscillation.

Target tracking suits gradual variation. Predictive or scheduled scaling should pre-warm capacity before a known sale starts. A sharp unpredictable spike still needs reactive scaling and excess warm capacity, because a new container or instance is not ready instantaneously.

Scale-in and back-pressure

Scale-in should be slower than scale-out: observe a cool-down period, finish in-flight requests, drain old connections and confirm replicas are removed from service discovery. Use bounded queues, rate limits, graceful retries with jitter and per-service timeouts.

Synchronous checkout flows need protection against retry storms. Circuit breakers, bulkheads, per-user quotas and a waiting room can preserve critical operations when external payment or inventory services become saturated.

Engineering evidence / comparison

Signal	What it reveals	Action if persistent
p95 checkout latency	User-visible response degradation	Scale web tier; profile DB and dependencies
Request rate / target	New demand arriving	Add replicas before queueing grows
Queue age / backlog	Workers cannot keep up	Scale workers; reduce nonessential jobs
5xx / timeout rate	Failed transactions or dependencies	Trigger incident; guard retries
DB connections / locks	Backend nearing saturation	Optimize queries and connection pools

CAPACITY PRINCIPLE
Capacity must be sized for the bottleneck tier. Adding web replicas cannot fix a database lock or a saturated payment gateway.

Scaling workflow and validation

ANALYSIS FOCUS / CLOSED-LOOP MONITORING AND RELIABILITY CHECKS

WORKFLOW / CONTROL DECISIONS



Branch: unhealthy replica -> remove from load balancer; overload -> throttle nonessential traffic.

FIGURE / Operational decision flow - illustrates principal dependencies and control flow; provider-neutral design.

Monitoring and alert design

Centralized telemetry combines infrastructure metrics (CPU, memory, network), service indicators (RPS, p95 latency, errors), distributed traces and checkout business events. Alerts should distinguish genuine customer impact from temporary resource spikes.

Use dashboards comparing desired versus available replicas, auto-scaler decisions and startup times. Link alerts to an on-call runbook that includes manual limits, safe rollback, CDN configuration and checkout protection mechanisms.

Failure-injection verification

Test unexpected replica termination, load balancer health-check failure, loss of one availability zone, database overload and payment gateway latency. During each test, observe recovery time, user-facing errors and whether duplicate orders were prevented.

Test both rising and falling demand. A service that scales out correctly but terminates sessions during scale-in has not met the user-experience requirement.

Engineering evidence / comparison

Test	Expected result
Kill active web replica	Healthy replicas serve requests; failed endpoint stops receiving traffic
Introduce 3x traffic within minutes	Replica count grows within configured limits; latency remains bounded
Disable an availability zone	Traffic moves to surviving zones with sufficient tested capacity
Repeat payment callback	Idempotency prevents duplicate capture / order creation

Question 1

Applied seasonal-sale example and recommendation

ANALYSIS FOCUS / WORKED SIZING, RISKS AND PRACTICAL JUDGMENT

Illustrative workload calculation

Assume peak checkout and shopping API demand reaches 1,800 requests per second. Suppose one tested application replica can sustain 250 requests per second at the chosen latency target. Minimum healthy replicas = ceiling(1,800 / 250) = 8; this figure is an illustrative calculation, not a measured capacity claim.

For a three-zone deployment tolerant of one complete zone outage, 12 replicas distributed 4 per zone leave 8 healthy replicas after one zone fails. This is only adequate if the database, network, cache and payment dependencies can also sustain the peak, and the remaining zones have resource headroom. Pre-warm replicas before the sale.

Real-world e-commerce flow

Before a festival promotion, marketing traffic can increase product searches many-fold while actual orders grow more slowly. Cache product images and public catalog views, keep inventory and price decisions authoritative, and prioritize checkout traffic over recommendations or analytics.

After demand recedes, gradually scale down application replicas and workers. A post-event review should compare forecast load with real demand, checkout completion, abandoned carts, autoscaling lag and total spend.

Costs, residual risks and conclusion

Main cost drivers are replica-hours, managed database capacity, cache nodes, data transfer, log ingestion and CDN requests. Control spend with realistic scaling minima, budgets and scheduled capacity; never sacrifice required redundancy simply to reduce idle cost.

The recommended solution combines pre-warmed multi-zone capacity, health-aware load balancing, auto-scaling, CDN caching, queue isolation and SLO-driven monitoring. It improves resilience when each critical dependency has a corresponding recovery plan.

Engineering evidence / comparison

Risk	Mitigation / evidence
Cold-start or slow instance warm-up	Maintain minimum spare replicas; load test warm-up
Retry storm at payments	Idempotency + jitter + circuit breaker
Single-zone capacity shortfall	Fault-inject AZ failure under realistic peak load
Runaway infrastructure bill	Scaling limits, budgets, alerts and demand review

ASSESSMENT CONCLUSION
Cloud elasticity reduces downtime when scaling signals, healthy routing, backend limits and cross-zone capacity are treated as one system rather than independent features.

Question 2

Storage durability, redundancy and recovery objectives

QUESTION 2 An organization wants to store large volumes of backup data securely in the cloud. Discuss how cloud storage solutions ensure durability and data redundancy. How can encryption and access control protect sensitive backup data? Explain the cost advantages compared to traditional storage systems. Provide an example of a disaster recovery use case.

ANALYSIS FOCUS / BACKUP REQUIREMENTS AND PROTECTION STRATEGY

Scenario and objectives

The organization must retain large volumes of recoverable backup data while preventing accidental deletion, ransomware encryption, unauthorized access and hardware failure. Backup integrity and restoration speed matter as much as raw storage capacity.

Durability is the likelihood stored data remains intact; availability is whether a service can currently be reached; recoverability is whether an acceptable application state can be restored. None of these terms is interchangeable. Multiple replicas of corrupted data still contain the corruption.

Protection layers

Use snapshots or native database backups with application-consistent recovery points, transaction logs for point-in-time restore where supported, versioned object storage and separately administered immutable copies. Maintain a defined retention policy and isolate at least one backup copy from routine production credentials.

A practical interpretation of 3-2-1-1-0 is: three copies of important data, two storage types or failure domains, one offsite, one offline or immutable copy, and zero unverified restore errors. Validate each layer with actual recovery tests.

Recovery targets

RPO is the maximum acceptable loss of recent work measured as time. RTO is the maximum acceptable duration until service restoration. Both must be agreed by the business, then implemented with backup frequency, logging, automation and reserved recovery capacity.

Engineering evidence / comparison

Threat	Required safeguard	Proof
Disk / node failure	Provider redundancy and scrubbing	Object verification and service health
Accidental deletion	Versioning, soft delete, retention	Restore deleted test dataset
Ransomware / credential misuse	Immutability and separate access domain	Denied delete test + clean restore
Region outage	Replicated off-region copy	Region failover exercise

KEY DISTINCTION
Multi-zone replication increases resilience but is not a substitute for versioned, independently protected backups.

Question 2

Protected backup reference architecture

ANALYSIS FOCUS / ENCRYPTED, IMMUTABLE AND GEOGRAPHICALLY ISOLATED COPIES

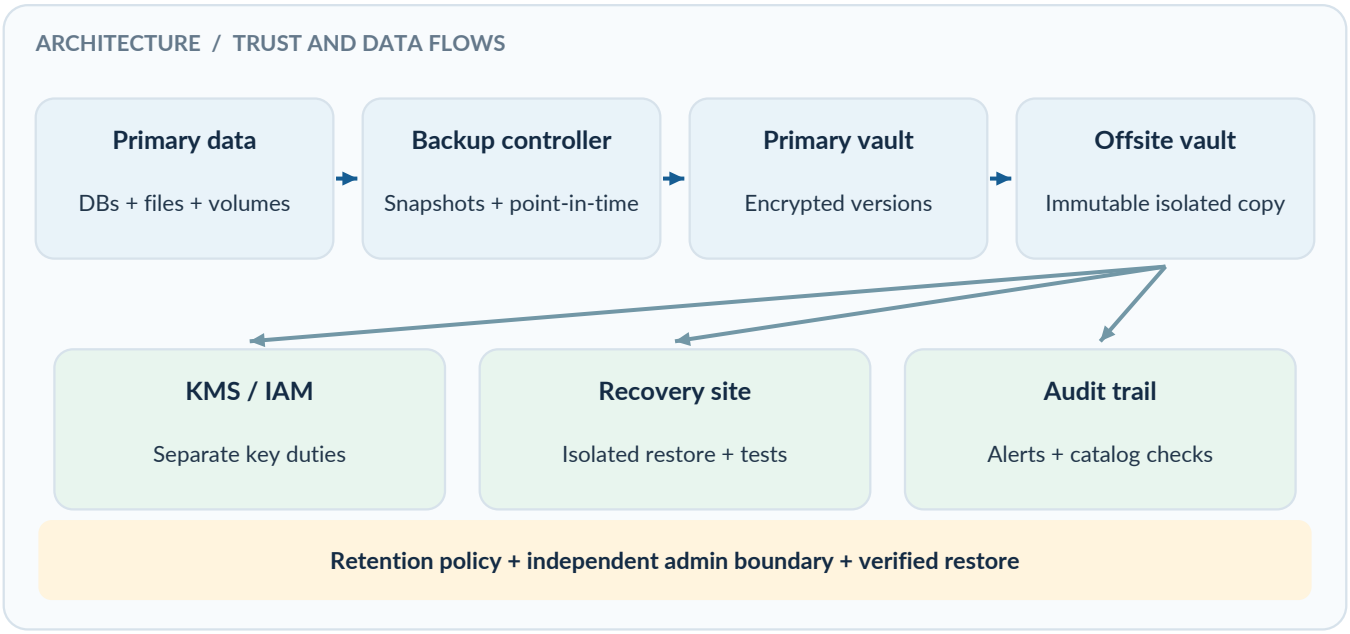


FIGURE / Cloud reference architecture - illustrates principal dependencies and control flow; provider-neutral design.

End-to-end data protection

Backup orchestration collects database snapshots, application files and virtual machine recovery points. These are encrypted and cataloged in a primary vault with retention and lifecycle rules. A distinct security boundary receives an immutable copy, ideally under separate administrative privileges.

Customer-managed cryptographic keys, privileged access logs, retention locks and restore permissions are configured independently. An isolated recovery environment provides a way to test backups without exposing the production network.

Metadata and consistency requirements

Backup metadata should capture source, timestamps, checksum, backup set, encryption key reference, retention class, job status and application consistency level. A backup that completes without error can still be unusable if its database transaction log chain is incomplete.

Cross-region copies strengthen recovery from a regional outage but incur storage and transfer charges and may be restricted by data-residency obligations. Choose replication destinations only after security and legal review.

ARCHITECTURE NOTE
The immutable vault and recovery account must not share unrestricted delete permissions with the production administrator.

Question 2

Encryption, access control and cost optimization

ANALYSIS FOCUS / SHARED RESPONSIBILITY AND LIFECYCLE DESIGN

Data confidentiality controls

Encrypt data in transit with authenticated TLS and at rest using supported cloud storage encryption. For sensitive backups, customer-managed KMS keys permit explicit access policies and audit trails. Key rotation, secure key recovery and deletion procedures must be tested.

Grant backup agents write-only or narrowly scoped roles where possible; restrict restore, copy, key-use, retention modification and permanent deletion to separate approved roles. Enable MFA or privileged-access approval for high-impact operations. Never embed cloud secrets in scripts.

Cost design

Use storage classes matched to required retrieval speed and frequency. Recently created backups may need fast access; old compliance archives can move to lower-cost tiers once restore-time requirements permit. Compression, incremental backups and deduplication reduce storage if compatible with application consistency.

A realistic total-cost model includes ingestion, stored GB-months, replication, API calls, retrieval, egress, key operations, monitoring, staffing, audit and periodic recovery tests. Deep archive is inexpensive to retain but may be slow or costly to retrieve.

Engineering evidence / comparison

Cost lever	Benefit	Tradeoff to validate
Incremental snapshots	Store changed blocks rather than repeated full copies	Restore chain dependency and corruption handling
Lifecycle tiering	Lower GB-month storage expense	Retrieval delay / minimum retention costs
Compression and deduplication	Less stored data and transfer	CPU overhead and limited gains on media
Automated pruning	Prevents uncontrolled growth	Must respect holds, compliance and rollback windows

SECURITY DESIGN
Use separate identities for backup creation, backup deletion, key administration and disaster-recovery approval. Separation of duties reduces correlated compromise.

Question 2

Disaster recovery workflow and test evidence

ANALYSIS FOCUS / RESTORE VALIDATION, RPO / RTO AND CONTINUITY

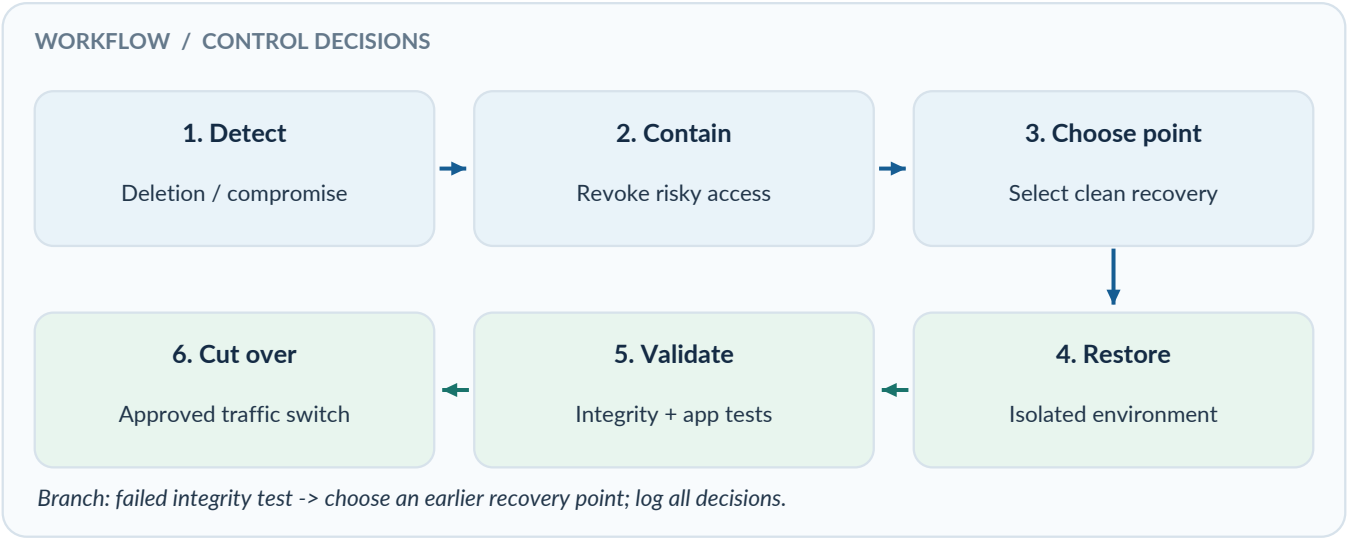


FIGURE / Operational decision flow - illustrates principal dependencies and control flow; provider-neutral design.

Recovery sequence

Classify the incident and prevent further destructive writes. Identify the last known-good backup and verify whether it meets the business RPO. Restore data into an isolated network, verify checksums and consistency, run malware or integrity assessments, then execute functional tests.

Only after validation should operators authorize traffic cutover and reopen business access. Document DNS updates, application secret rotation, database replay, user notifications, rollback procedures and evidence preservation.

Testing and measurement

Quarterly or risk-adjusted exercises should include deleted data, encryption-key denial, restore to a new account, region unavailability and backup catalog failure. Measure start-to-ready time rather than the moment the restore request was submitted.

A successful disaster recovery plan assigns named owners and includes exact steps for identity recovery and decryption-key access. Without keys and credentials, stored encrypted copies may be unrecoverable.

Engineering evidence / comparison

Validation step	Pass criterion
Backup retention	Policy protects backups for documented required duration
Immutable delete attempt	Unapproved delete or lock modification denied
Restore consistency	Application checks and referential integrity pass
RPO / RTO exercise	Observed loss window and recovery time meet agreed objectives

Question 2

Disaster scenario, worked examples and recommendation

ANALYSIS FOCUS / BUSINESS CONTINUITY WITH VERIFIABLE RECOVERABILITY

Worked disaster recovery example

Consider a payroll database damaged by accidental deletion at 14:20. Suppose the approved RPO is 60 minutes and RTO is four hours. With a verified 14:00 recovery point and intact log chain, the team could restore to 14:00 and lose at most 20 minutes of new activity; logs might permit even less loss.

If detection begins at 14:25, the restored environment is validated at 16:05 and the service is usable at 16:20, observed recovery time from incident at 14:20 is two hours. Both objectives would be met in this illustrative example. Such values are targets and examples, not claims about an actual provider.

Illustrative cost comparison method

Suppose a protected archive uses 20 TB. At a hypothetical storage price of USD 0.010 per GB-month and 20,000 GB assumed for transparent arithmetic, the storage line item is USD 200 per month; add version retention, replication, access, KMS and recovery-test fees separately. The assumption is for calculation only, not a current vendor quote.

Traditional storage additionally requires hardware purchase, rack space, power, replacement capacity, support and offsite custody. Cloud storage changes the spending pattern to consumption-based charges, but unexpectedly large egress or retrieval can reduce the savings.

Residual risks and final recommendation

Replication can propagate bad updates; stolen keys can defeat confidentiality; and retention locks may conflict with erasure obligations. Maintain incident-specific restore points, legal retention schedules, tested key custody and immutable offsite copies.

The recommended architecture combines encryption, least-privilege backup administration, versioning, isolated immutable copies, lifecycle tiering and evidence-based restore exercises. It protects confidentiality, integrity and business continuity together.

Engineering evidence / comparison

Review question	Acceptable evidence
Can we restore after account compromise?	Independent vault access + restore drill
Can we meet recovery time?	Timed end-to-end runbook exercise
Can auditors trace deletions?	Immutable administrative event logs
Can data remain in required jurisdiction?	Region/replication policy and access review

Question 3

CDN fundamentals and global latency requirements

QUESTION 3 A mobile app company plans to deploy its application globally. Explain how content delivery networks (CDNs) improve performance and reduce latency. How do edge servers enhance user experience across regions? Discuss how cloud providers support global availability. Relate this to a real-time streaming or gaming application.

ANALYSIS FOCUS / EDGE DELIVERY FOR A GLOBALLY DEPLOYED MOBILE APPLICATION

Application context

A single origin region forces distant users to traverse long network paths. This increases round-trip delay, makes static assets slower, and can amplify origin overload during major live events. A content delivery network places caching and delivery infrastructure nearer to end users.

CDNs are most effective for cacheable files such as images, application bundles, video segments and downloadable assets. Live multiplayer state, payments and rapidly changing personalized data generally need regionally deployed application servers or specialized edge compute, not blind HTTP caching.

Core CDN mechanisms

Edge points of presence (PoPs) receive client requests using anycast or latency-aware routing. A cache hit returns eligible content directly; a cache miss fetches the asset from an origin or origin shield and may populate the local cache according to HTTP caching policy.

TLS termination, connection reuse, HTTP/2 or HTTP/3 support, compression and optimized transport can improve perceived speed. Benefits depend on routing, object size, hit ratio, network conditions and the origin's cache rules.

Quality-of-experience targets

Define p50/p95 startup time, rebuffer ratio, video bitrate stability, edge hit ratio, origin egress, error rate and availability by region. For interactive games also monitor end-to-end input latency, jitter, packet loss and regional session placement.

Engineering evidence / comparison

Content class	Recommended delivery
Images / JS / CSS	Long-lived versioned CDN cache where safe
VOD video segments	CDN caching + ABR manifests and signed access
Live streaming segments	Short TTL or live-aware edge distribution
Game state / live interaction	Regional real-time servers, not ordinary static caching
Sensitive personalized APIs	No-store or carefully authenticated edge caching

DESIGN RULE
Choose delivery treatment per content class. Caching user-specific responses without correct keys and authorization can expose private content.

Question 3

Global streaming delivery reference architecture

ANALYSIS FOCUS / ORIGIN, TRANSCODING, EDGE AND REGIONAL REAL-TIME SERVICES

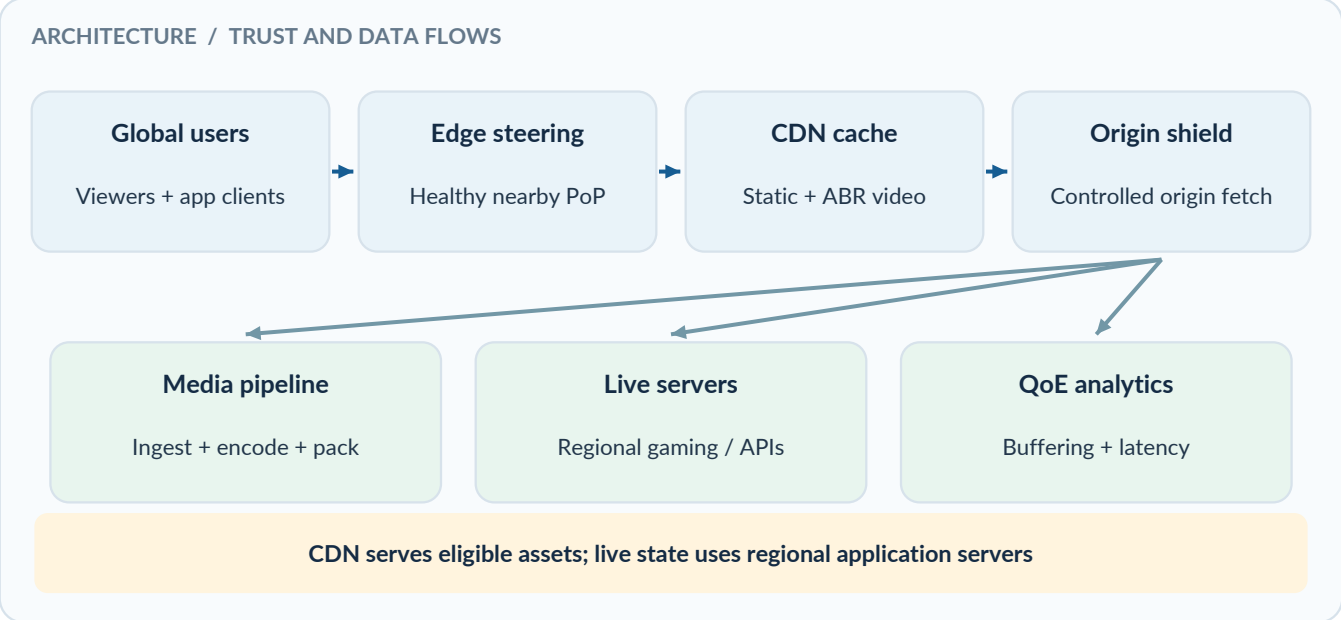


FIGURE / Cloud reference architecture - illustrates principal dependencies and control flow; provider-neutral design.

Content distribution path

Live video ingestion feeds encoders and packagers that create adaptive-bitrate representations and segments. The origin stores or serves those assets; an origin shield reduces duplicate cache misses before they reach the origin. Edge PoPs distribute segments to nearby users.

Latency-aware traffic steering directs sessions toward healthy locations. Regional real-time servers process game state or other interactive operations where milliseconds of extra round-trip time matter. Those interactions should use appropriately authenticated stateful protocols rather than a generic static CDN cache.

Regional availability and fallback

Providers can place origins, delivery points and application clusters across multiple regions. Health checks, independent DNS/routing, replicated content and a planned failover procedure reduce regional dependencies. Global deployment also requires users' permitted data locations and edge cache policies to be reviewed.

ARCHITECTURE NOTE
CDN PoPs primarily accelerate asset distribution; regional game or API servers separately handle changing application state.

Question 3

Cache policies, edge security and performance math

ANALYSIS FOCUS / CORRECT CACHE KEYS AND MEASURED OPTIMIZATION

Caching policy and invalidation

Use versioned asset URLs and suitable Cache-Control headers. Cache keys should include only parameters that affect the representation, while varying or bypassing caches for authorization context, language and other required dimensions. Avoid putting secret tokens in cache keys or logged URLs.

When a live asset changes, shorten TTL appropriately or use targeted invalidation. A very short TTL increases origin traffic; a long TTL risks stale content. Origin shielding, stale-while-revalidate where supported and request collapsing help control spikes.

Performance calculation

If 100,000 asset requests are received and 92,000 are served from edge caches, the observed request hit ratio is 92%. Only 8,000 reach the next tier, assuming no other bypass requests. Bandwidth offload can differ from request hit ratio because object sizes vary.

Suppose a 2 MB video segment requires 150 ms to first byte from a distant origin but only 25 ms from a local edge under a particular measured path. That illustrative first-byte improvement is 125 ms; complete playback gain depends on throughput, player buffer and additional segment requests.

Edge security boundaries

Protect content with signed URLs or cookies, token expiry, HTTPS, WAF rules and geo-policy only where legally appropriate. A signed URL validates access to an asset; it does not replace full entitlement checks or protect content once legitimately downloaded.

Engineering evidence / comparison

CDN metric	Meaning	Operational decision
Request hit ratio	Fraction served without an origin fetch	Improve cache keys / TTL / asset versioning
p95 first-byte latency	Tail network and edge response delay	Inspect PoP routing and origin shielding
Rebuffer ratio	Playback interruption relative to view time	Investigate throughput and ABR behavior
Origin 5xx / saturation	Backend insufficient or unhealthy	Failover and protect origin from floods

Question 3

Cache-miss workflow and geographic test plan

ANALYSIS FOCUS / ROUTING, CACHING, VALIDATION AND USER EXPERIENCE

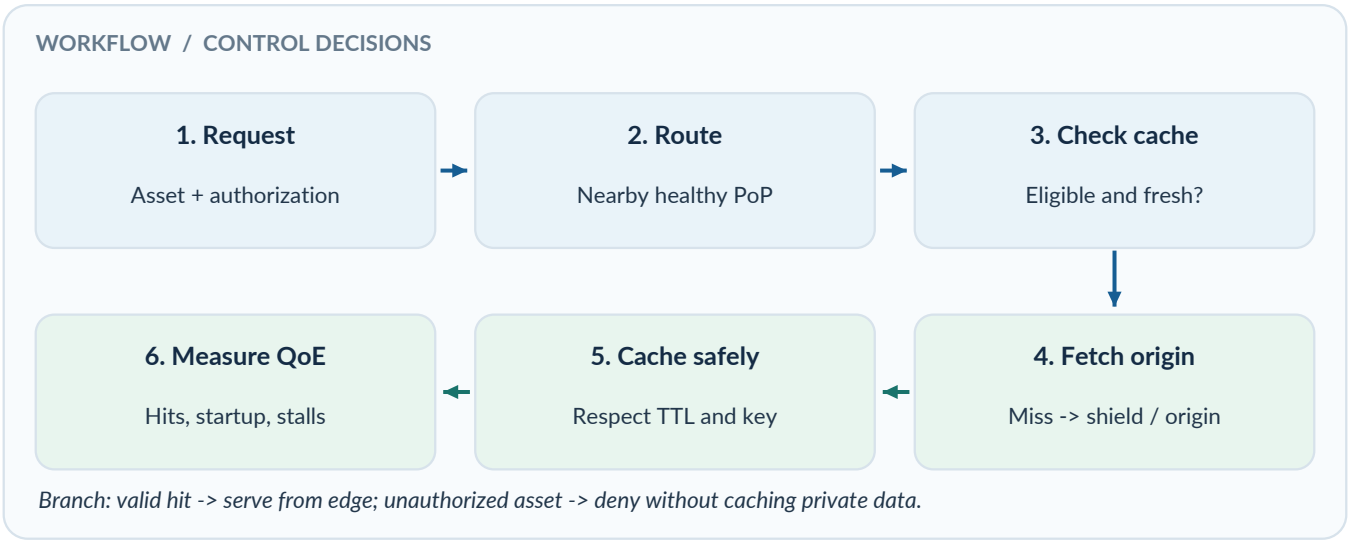


FIGURE / Operational decision flow - illustrates principal dependencies and control flow; provider-neutral design.

End-to-end operation

A request is routed to a nearby healthy edge location. The cache policy determines whether the object is eligible and fresh. A hit is returned locally; a miss goes through an origin shield or the origin, then may be cached for subsequent eligible requests.

Telemetry measures edge hits, misses, bytes served, performance by geography and content type. Poor outcomes can arise from underperforming origins, cache fragmentation, transport problems or an overly large player buffer; diagnosing only the CDN hit rate is insufficient.

Multi-region validation

Test from representative locations, mobile network conditions and device classes. Exercise regional routing changes, partial CDN failures, expired signed URLs, origin health changes and cache invalidation. Include real-time interactive tests separately from streaming playback.

Engineering evidence / comparison

Test scenario	Pass evidence
Cacheable static asset, second request	Eligible edge hit observed without cross-user leakage
Live playlist refresh	Updated content appears within intended freshness window
Primary origin outage	Configured secondary origin responds after tested failover
Token expired / user unauthorized	Access denied and event logged
Users on weak mobile networks	ABR adapts while minimizing playback stalls

Question 3

Practical media and gaming case studies

ANALYSIS FOCUS / ADAPTIVE STREAMING, LIVE GAMES AND TRADEOFFS

Streaming application

A mobile live sports platform creates 240p, 480p, 720p and 1080p renditions and packages short stream segments. Users fetch a manifest and segments from the nearest acceptable PoP. Adaptive-bitrate logic selects a representation based on measured bandwidth and buffer health.

During a popular event, a 95% byte offload would mean only 5% of eligible byte volume is fetched upstream, under that example's accounting definition. This can reduce origin load and improve scalability but raises CDN delivery and cache-management costs.

Gaming application

A global game uses CDN delivery for installers, textures, patches and replay clips. Real-time match messages go to nearby authoritative game servers via low-latency routing, since state changes frequently and must be ordered and validated. Matchmaking considers regional latency, player population and compliance boundaries.

Edge acceleration is not a universal solution: UDP and gaming transport support, authoritative simulation, state reconciliation and anti-cheat controls require specialized implementation.

Cost and architecture recommendation

Estimate delivery spend from requests, bytes, origin transfer, transcoding, storage, regional traffic and observability. Use real traffic distribution to compare CDN and origin costs; prohibit unnecessary duplicate cache variants and control expensive invalidations.

The best architecture combines CDN caching for eligible assets, regional live services for dynamic interaction, multi-region origin redundancy, secure cache rules, telemetry and continuous playback/latency testing.

Engineering evidence / comparison

Design tradeoff	What to optimize
Short versus long TTL	Freshness against origin demand
More CDN edge reach	Latency versus delivery cost
Regional game servers	Player delay versus operating footprint
Multi-CDN failover	Resilience versus configuration complexity

FINAL RECOMMENDATION
Match content type to delivery path: cache stable objects close to users, process live state in the right region, and prove gains with end-user QoE metrics.

Question 4

Compliance obligations and cloud security model

QUESTION 4 A financial institution requires strict compliance and data security in cloud usage. Explain how cloud providers support regulatory compliance and auditing. What role do logging, monitoring, and encryption play in security? How can the company ensure data sovereignty requirements are met? Give an example involving sensitive financial transactions.

ANALYSIS FOCUS / SHARED RESPONSIBILITY, REGULATED DATA AND AUDIT READINESS

Scenario interpretation

A financial institution processes highly sensitive identity, account and transaction data. Cloud adoption requires enforceable security controls, an auditable evidence trail and explicit handling of data-location restrictions. Provider certifications can support assurance, but the institution remains responsible for its own data classification, configurations and applicable legal obligations.

Compliance controls must be mapped to the jurisdictions and regulations actually applicable to the institution. A cloud provider cannot guarantee an institution's compliance merely by advertising certifications. Requirements for storage, processing, backup, logging, support access and cross-border transfer may differ.

Responsibilities and trust boundaries

The provider generally secures the underlying cloud infrastructure according to the service model. The customer controls identity, data access, application code, encryption configuration, retention and most workload-specific monitoring. In managed services, the exact division depends on the contract and product.

Enforce least privilege, strong authentication, change approval, network segmentation and separation of duties for financial transactions. Treat administrators, build systems and third-party integrations as distinct risk categories.

Control objectives

Preserve confidentiality through encryption and access minimization; integrity through authenticated transactions and auditability; availability through redundant regions or zones as permitted; and accountability through tamper-evident logs and review processes.

Engineering evidence / comparison

Control objective	Evidence of implementation
Identity assurance	MFA policies, privileged access review, session logs
Data confidentiality	Approved encryption, key access control, tokenization
Auditability	Time-synced immutable logs and change history
Location governance	Region policies, backup residency, transfer review
Operational resilience	Restore drills, incident response and vendor risk review

COMPLIANCE PRINCIPLE
First specify the applicable laws and contracts. Then map each requirement to a responsible owner, technical control, retained evidence and a review cycle.

Question 4

Secure financial-cloud reference architecture

ANALYSIS FOCUS / IDENTITY, ENCRYPTION, TRANSACTION AND AUDIT BOUNDARIES

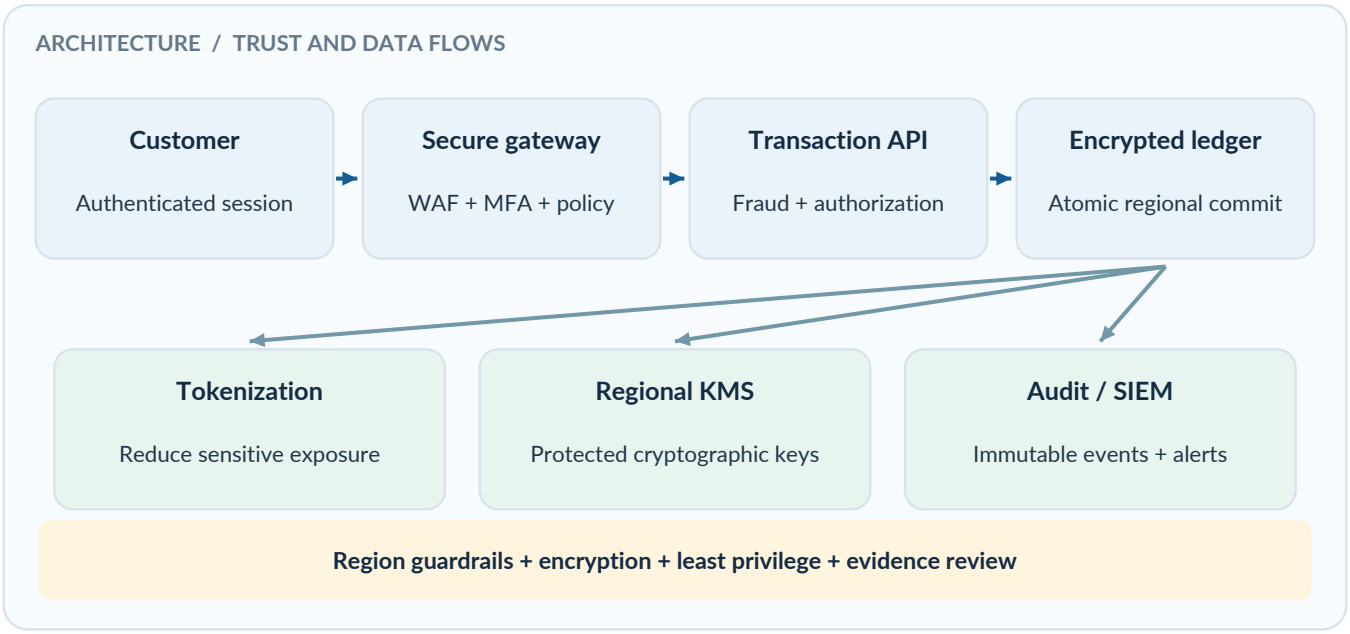


FIGURE / Cloud reference architecture - illustrates principal dependencies and control flow; provider-neutral design.

Protected transaction pipeline

Applications use an authenticated gateway and scoped authorization. The transaction service validates permissions and input, applies fraud rules, and commits funds movement atomically using database constraints and idempotency keys.

Sensitive identifiers may be tokenized before reaching less-trusted services. Encryption keys remain under tightly restricted KMS/HSM administration. Logs capture business events and administrative actions without unnecessarily exposing card data, secrets or full identity details.

Audit and evidence architecture

Events are timestamped and collected centrally with controlled write access, defined retention, integrity checks and monitoring. Security analytics detect unusual transfers, privilege changes and suspicious access. Investigators need searchable, well-scoped evidence while the institution follows its privacy and retention requirements.

Region allowlists, private network endpoints and explicit cross-region data-flow approvals help prevent accidental movement of regulated datasets. Contracts and subprocessors must still be assessed for support and operational access paths.

ARCHITECTURE NOTE
Logical restrictions on region placement are necessary but not sufficient to establish legal data sovereignty; assess administrators, support access, logs and backups as well.

Question 4

Encryption, monitoring and evidence mapping

ANALYSIS FOCUS / PREVENTIVE, DETECTIVE AND CORRECTIVE SECURITY

Cryptographic controls

Use TLS with sound certificate management for data in transit and strong encryption for stored databases, snapshots, backups and logs. Customer-managed keys offer additional control where required, but they create availability and operational responsibilities for rotation, recovery and decommissioning.

Prefer tokenization or masking for fields that services do not require in clear text. Protect secrets through managed secret storage and short-lived workload identities rather than static environment credentials.

Security telemetry and operations

Combine audit trails, application transaction logs, operating system signals, identity events and network controls. Standardize timestamps and event identifiers, then route relevant signals to a SIEM or equivalent analysis platform.

Detection rules should identify anomalous beneficiary updates, repeated failed MFA attempts, privileged permission escalation, unusual exports and unexpected cross-region transfers. Alert severity should reflect transaction and data impact; investigations must preserve evidence and avoid over-collecting personal data.

Data location and access governance

Build an inventory of sensitive fields and data stores, including logs, queues, ML datasets, backup archives and support tickets. Constrain allowed cloud regions through organizational policy and deployment guardrails. Validate actual data movement through audits and provider documentation.

Engineering evidence / comparison

Security layer	Required practice	Validation
In transit	Authenticated TLS and approved ciphers	Protocol scans and certificate audit
At rest	Database/volume/object encryption	Configuration and key-access review
Administrative identity	MFA + just-in-time or least privilege	Negative privilege test
Audit trail	Tamper resistance and defined retention	Attempt unauthorized modification
Sovereignty	Residency policy and reviewed transfer paths	Geo-placement and data-flow audit

Question 4

Transaction security workflow and test strategy

ANALYSIS FOCUS / ATOMICITY, AUTHORIZATION AND DETECTIVE MONITORING

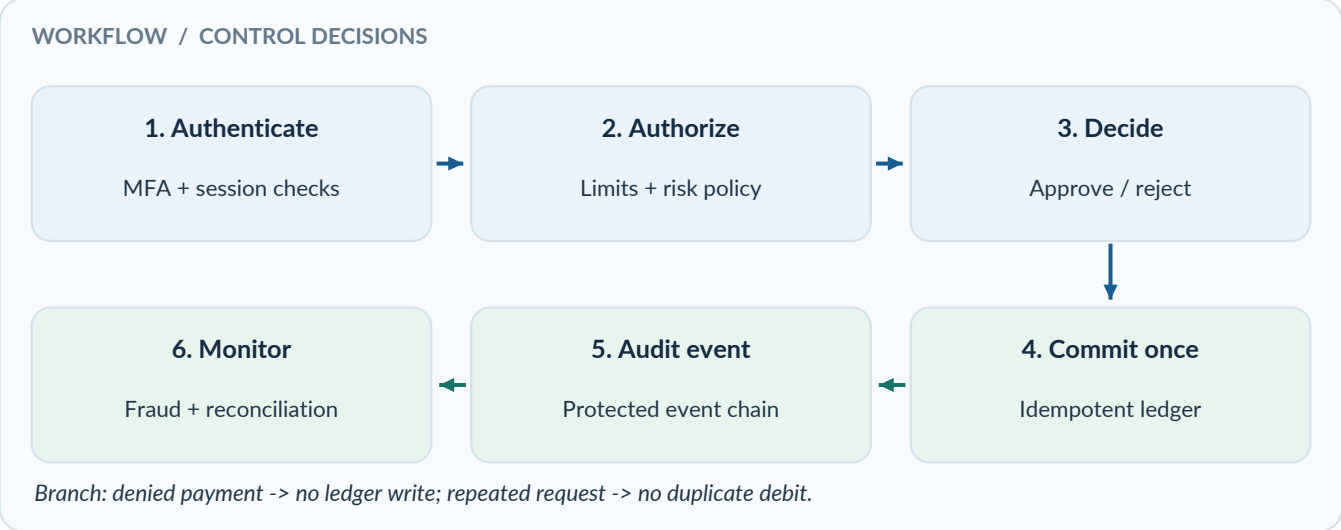


FIGURE / Operational decision flow - illustrates principal dependencies and control flow; provider-neutral design.

Sensitive funds-transfer workflow

After MFA and session verification, the service validates amount, beneficiary, account balance and risk signals. The authorization layer enforces transaction privileges. An idempotent transaction commits the ledger update once; events are recorded in a controlled audit store.

A denied transfer must not change balances. A retry must not produce a duplicate debit. An attacker should not be able to erase the audit trail or move logs outside approved regions.

QA and security verification

Test positive and negative authorization, replay protection, expired sessions, malformed input, role escalation, key policy denials, logging completeness, region restrictions and recovery from partial failure. Assess monitoring against both known attack simulations and benign business bursts.

Define evidence retention before testing. Logs that contain secrets or unnecessary personal data may cause a new compliance problem even if the original transaction was secured.

Engineering evidence / comparison

Test	Expected secure result
Unauthorized amount update	Request rejected with audit event
Duplicate transaction request	Single ledger commit using idempotency key
Unauthorized cross-region backup	Deployment or transfer policy blocks action
Key-use denial	Decryption fails safely and event is logged
Privileged admin change	Approval + separate traceable audit record

Question 4

Practical bank transaction example and governance

ANALYSIS FOCUS / CONTROLS, INCIDENT RESPONSE AND FINAL JUDGMENT

Illustrative regulated-payment case

A customer initiates a high-value transfer from a mobile device. The institution authenticates the session, applies step-up MFA when policy requires it, checks transaction limits and beneficiary status, then executes an atomic ledger update. It records a correlation ID, decision, time, policy outcome and masked identifiers.

Security personnel investigate unusual transfer frequency using centrally collected logs and trace data. If the transfer is denied, the application returns a safe explanation; if authorized, downstream notification failures cannot cause duplicate ledger commitments.

Data sovereignty implementation

Suppose policy requires a particular country's approved region for account data, transaction records, backup copies and relevant audit logs. Enforce region allowlists and blocked replication destinations, then review disaster recovery, provider support access, analytics exports and subprocessors with the institution's legal and compliance teams.

This example deliberately does not assert a particular country's law. Data residency alone does not establish that all access or processing is legally permissible; requirements must be confirmed for the institution's jurisdiction.

Management and response controls

A defensible program includes vendor due diligence, signed architecture reviews, access recertification, key-rotation exercises, independent penetration testing, incident tabletop exercises, recovery drills, evidence sampling and recorded remediation ownership.

The recommended approach is defense in depth: approved data location, least-privilege identity, encryption and tokenization, atomic transaction controls, immutable logs, continuous monitoring and verified incident response.

Engineering evidence / comparison

Risk	Primary control
Credential theft	Phishing-resistant MFA + adaptive policy
Data leak by misconfiguration	Policy-as-code and region guardrails
Audit tampering	Write-restricted immutable log storage
Duplicate financial posting	Atomicity and idempotency controls
Regulatory scope mismatch	Legal requirements traceability and evidence review

ASSESSMENT CONCLUSION
Cloud tools provide security building blocks; the regulated institution remains accountable for correct configuration, independent validation and compliance-specific evidence.

Question 5

Containerization versus virtual machines

QUESTION 5 A development team wants to adopt containerization for application deployment. Explain how containers differ from virtual machines in terms of performance and resource usage. How does orchestration (like Kubernetes) simplify deployment and scaling? Discuss portability benefits across different cloud environments. Provide an example of a microservices-based application.

ANALYSIS FOCUS / EXECUTION ISOLATION, PERFORMANCE AND RESOURCE USE

Foundational distinction

A virtual machine runs a guest operating system above a hypervisor and virtualized hardware. A container packages an application and its user-space dependencies while sharing the host operating system kernel, isolated using namespaces and resource controls such as cgroups.

Containers are often smaller to distribute and faster to start than full guest operating systems, but results depend on image size, platform, workload and security controls. They are not a universal replacement for VMs: strong kernel isolation, mixed operating systems and regulated workloads may favor VM boundaries.

Resource and lifecycle implications

A container image consists of immutable, reusable layers; a VM image commonly includes a complete guest OS filesystem. Containers can use resources densely, but incorrect limits may cause CPU throttling, OOM termination, network saturation and noisy-neighbor problems.

Virtual machines remain a common host layer for managed Kubernetes clusters. Running containers on VMs can combine fleet management with the tenant isolation and lifecycle controls of a hypervisor.

Portable packaging

OCI-compatible images improve application packaging portability across cloud environments. Kubernetes manifests describe deployments and services in a common API, but storage, ingress, load balancers, identity, network policy and managed databases still vary across providers.

Engineering evidence / comparison

Dimension	Container	Virtual machine
Operating system	Shares compatible host kernel	Runs independent guest OS
Deployment unit	OCI image and runtime spec	Guest disk image + virtual hardware
Typical operational density	Often efficient for small services	More guest-OS overhead
Isolation boundary	Kernel namespaces / cgroups / policy	Hypervisor-separated guest
Best fit	Microservices, CI jobs, portable runtimes	Mixed OS, strong guest isolation, legacy apps

DECISION PRINCIPLE
Choose a combination: deploy portable containerized services on hardened, properly isolated VM worker nodes when that best fits security and operations.

Question 5

Kubernetes microservices reference architecture

ANALYSIS FOCUS / IMAGE LIFECYCLE, INGRESS, DISCOVERY AND RESILIENCE

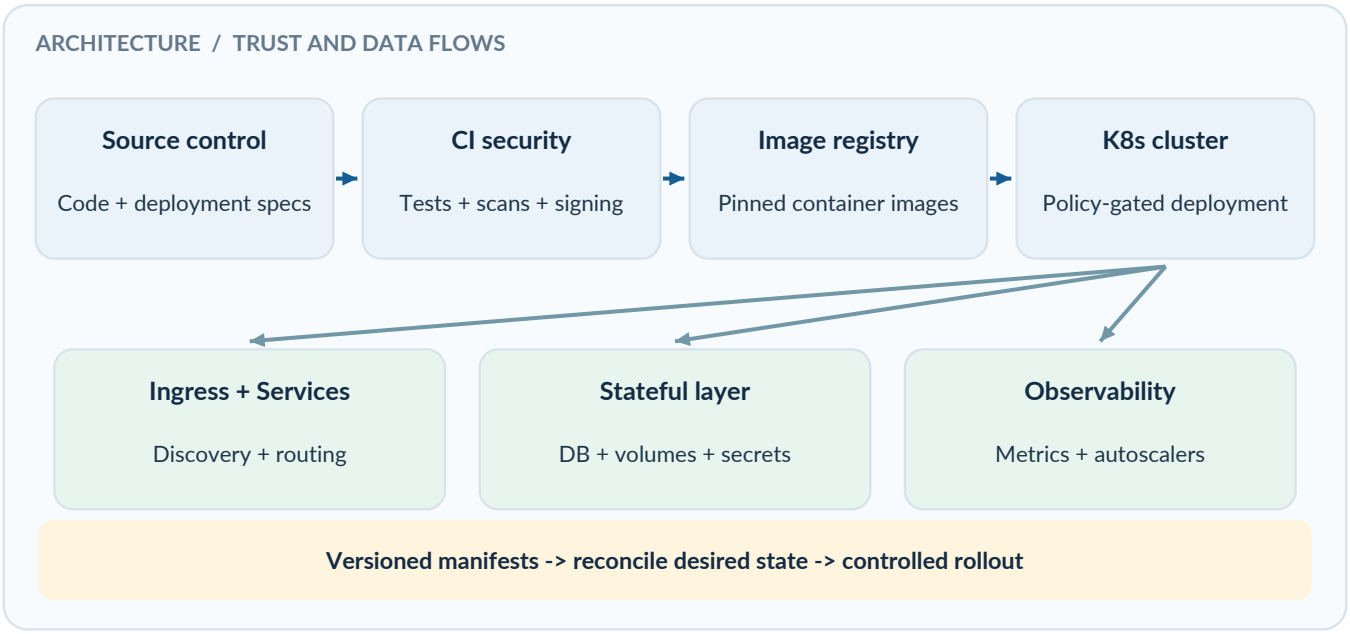


FIGURE / Cloud reference architecture - illustrates principal dependencies and control flow; provider-neutral design.

Deployment layers

Developers commit code and declarative deployment configuration. CI builds, tests and signs images, then publishes by immutable digest. A GitOps controller or delivery pipeline applies approved manifests to a Kubernetes cluster under policy checks.

External traffic reaches ingress and Kubernetes Services, which direct requests to healthy pods. Scheduler placement spreads replicas across worker nodes and availability zones where configured. Databases and other stateful systems use managed services or carefully operated persistent volumes.

Operations and failure boundaries

Readiness and liveness probes serve different purposes: readiness governs whether a pod should receive traffic; liveness can trigger restarts of unhealthy processes. Resource requests affect scheduling; resource limits bound usage. Both require load testing and tuning.

The monitoring layer collects container-level metrics, service latency, traces and security events. The platform must protect cluster credentials, registry access, admission policy and node patch levels.

ARCHITECTURE NOTE
A Kubernetes Service provides a stable virtual endpoint; ingress controls external HTTP(S) routing. Neither automatically guarantees an application can tolerate failed dependencies.

Question 5

Orchestration, auto-scaling and portability

ANALYSIS FOCUS / DECLARATIVE RECONCILIATION AND CONTROLLED ROLLOUT

Kubernetes building blocks

Deployments maintain the desired replica count and enable rolling changes. Pods provide the smallest deployable unit. Services create stable discovery addresses. ConfigMaps and Secrets hold configuration and sensitive values with appropriate access controls.

Horizontal Pod Autoscaler (HPA) adjusts replica counts based on supported resource, custom or external metrics. Cluster autoscalers add worker-node capacity when unscheduled pods require it. StatefulSets and persistent volumes support stateful services but do not replace database recovery plans.

Deployment and security policy

Use rolling or canary deployments, readiness gates, deployment history and automatic rollback when error budgets deteriorate. Apply image scanning, signed artifacts, minimal base images, non-root execution, RBAC, network policies and scoped service accounts.

An image that works on multiple registries may still depend on a cloud-specific load balancer or storage class. Portability should be tested with repeatable infrastructure-as-code, an abstracted secrets interface and automated service-level tests on each target environment.

Cost and resource sizing

Right-size CPU and memory requests from observations, not guesses. Scale replicas on service demand; set quotas to prevent one team from exhausting a shared cluster. Reserve enough spare node capacity for deployment surges and infrastructure failures.

Engineering evidence / comparison

Kubernetes resource	Primary role	Operational check
Deployment	Desired replicas and rollout	Health-gated rolling update
Service / Ingress	Discovery and external routing	TLS, route and failover test
HPA + cluster autoscaler	Pod and node scaling	Load and scheduling-pressure test
NetworkPolicy + RBAC	Limit service and human privilege	Denied east-west access test
PersistentVolume / external DB	Stateful storage	Backup and failover restore test

Question 5

CI/CD workflow, Kubernetes tests and rollback

ANALYSIS FOCUS / FROM SOURCE COMMIT TO SAFE PRODUCTION TRAFFIC

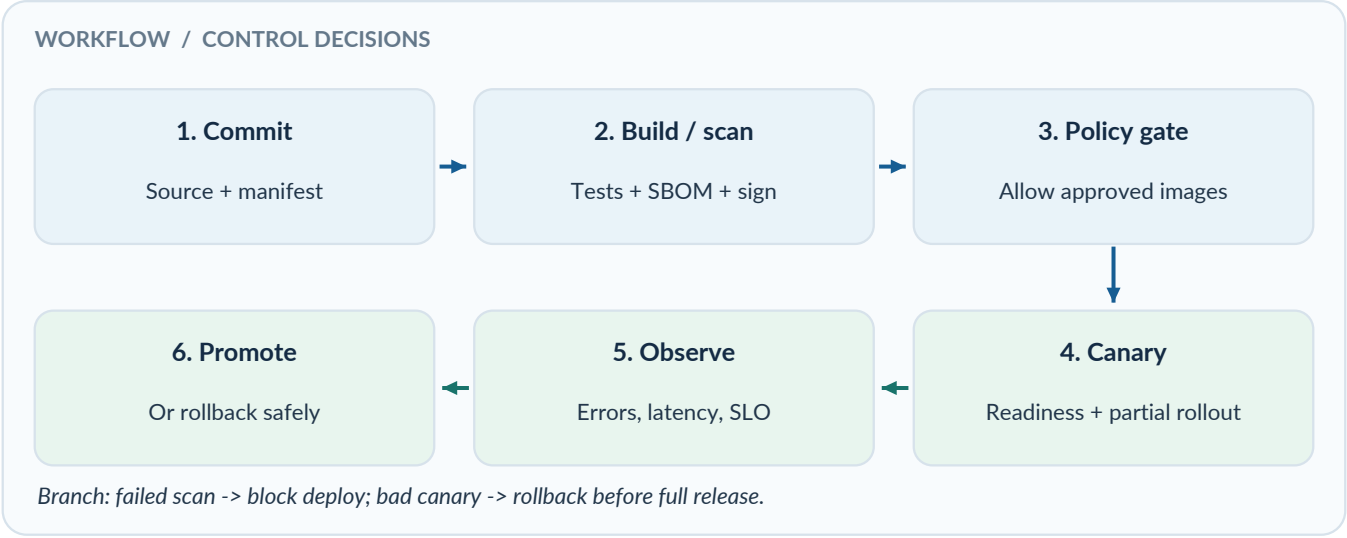


FIGURE / Operational decision flow - illustrates principal dependencies and control flow; provider-neutral design.

Continuous delivery lifecycle

Each change triggers compilation, unit tests, dependency and image scanning, artifact signing, and policy review. A staged deployment receives a small portion of traffic; telemetry determines whether it meets latency, error and availability gates before promotion.

Failed gates should stop progression or roll back automatically using a tested strategy. Rollback of application code does not automatically reverse incompatible database schema changes, so use backward-compatible migrations and controlled data rollback plans.

Load and resilience verification

Simulate a node outage, pod eviction, increasing CPU pressure, a poisoned image push, missing registry credentials and an unavailable downstream dependency. Verify that routing avoids unhealthy replicas, security controls deny prohibited deployments and monitoring captures user impact.

Engineering evidence / comparison

Scenario	Expected result
Single pod dies	Replacement scheduled; healthy replicas keep serving
Traffic doubles	HPA expands pods; nodes scale if needed
Image scan / signature fails	Admission or CI policy blocks deployment
Canary generates excess 5xx	Promotion halted and prior version restored
Cross-namespace request denied	Network policy blocks unauthorized traffic

Question 5

Microservices case study and final recommendation

ANALYSIS FOCUS / CLOUD PORTABILITY WITH REALISTIC OPERATIONAL TRADEOFFS

Illustrative e-commerce microservices

A shopping application has independent catalog, cart, user, order, inventory, payment and notification services. The catalog service scales for browsing, while checkout and inventory prioritize consistency. Service discovery lets frontends use stable names rather than fixed pod IP addresses.

Notification work can run asynchronously through a queue. Order and payment operations use idempotency to tolerate retries. The system traces a customer request across services with correlation IDs and monitors p95 latency and error rate separately per service.

Application portability across providers

Build OCI images once and promote signed digests across environments. Keep Kubernetes manifests mostly portable using overlays for ingress controllers, storage classes and provider identity. Test recovery and traffic routing when moving from one provider to another.

Portability is a spectrum: application packaging is comparatively standardized, but networking, persistent storage, observability products, policy integrations and data transfer may require provider-specific adapters. Avoid promising zero-cost migration.

Final design judgment

Use containers for independently deployable services, Kubernetes for scheduling and reconciliation, GitOps and policies for controlled changes, and a hardened VM/node substrate. Adopt strong monitoring and a migration test suite before claiming multi-cloud readiness.

The desired outcome is faster repeatable delivery, efficient resource use and failure containment, while preserving security, application correctness and predictable operations.

Engineering evidence / comparison

Failure or limitation	Recommended response
One microservice floods dependencies	Quotas, rate limits, circuit breakers
Incompatible database schema	Expand-and-contract migrations
Single cluster unavailable	Multi-zone or multi-cluster recovery plan
Different clouds expose different APIs	Portable core manifests + provider overlays
Compromised image in registry	Signature verification and admission control

ASSESSMENT CONCLUSION
Containerization makes software packages repeatable; orchestration makes running them manageable. Portability and reliability still require disciplined state, identity, network and recovery engineering.

ASSESSMENT RUBRIC

Original grading criteria from the uploaded CSA1516 CO2-AT3 assessment

Application Based Questions Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Relevance to Application	25%	Response directly addresses the given use case with strong relevance	Mostly relevant response	Partially relevant	Weak relevance
Use of Cloud Concepts	25%	Applies appropriate concepts accurately	Mostly accurate application	Basic application only	Incorrect concept usage
Practical Insight	20%	Shows strong real-world understanding	Shows reasonable practical understanding	Limited practical insight	No practical insight
Completeness	15%	Response covers all required aspects	Covers most aspects	Covers only basic aspects	Incomplete response
Clarity	15%	Clear and concise answer	Generally clear	Some confusion in expression	Poor clarity

Reproduced directly from the source PDF to preserve the published descriptors and weightages.